Task 7 – Merge & Resolve Merge Conflicts

Git & GitHub Fundamentals (Industry Workflow) | Estimated Time: 1 Hour

---

What's Going On Here?

You've been working on separate branches — now it's time to bring that work back together. Merging is how you take commits from one branch and fold them into another. Most of the time it works smoothly. But sometimes two branches changed the same line in different ways, and Git can't decide which version to keep — that's a merge conflict. This task teaches you both: the easy merge and the tricky one.

Step 1 – Merge One Branch Into Main

The golden rule: always switch to the branch you want to merge INTO first. Since you want feature-login's work to land in main, go to main first:

```
git switch main
git merge feature-login
```

If everything went smoothly, Git will print something like 'Fast-forward' or show a summary of files changed. That means there were no conflicts — Git just moved main forward to include all of feature-login's commits. Nice and clean.

> *Fast-forward happens when main hasn't moved since you branched off — Git simply slides the pointer forward. If main had new commits too, Git creates a 'merge commit' instead, combining both histories.*

Step 2 – Create an Intentional Conflict

Now you're going to deliberately cause a conflict so you can learn how to fix it. The idea is simple: edit the same line of the same file on two different branches, then try to merge. Git won't know which version you want — and that's exactly what we want to practise.

First, edit notes.txt on main:

```
git switch main
echo "Main branch update" > notes.txt
git add notes.txt
git commit -m "Update notes.txt on main"
```

Now switch to feature-profile and edit the same file differently:

```
git switch feature-profile
echo "Feature-profile branch update" > notes.txt
git add notes.txt
```

```
git commit -m "Update notes.txt on feature-profile"
```

Now go back to main and try to merge:

```
git switch main
git merge feature-profile
```

Git will stop and tell you there's a conflict in notes.txt. It can't continue until you decide what the file should look like.

Step 3 – Resolve the Conflict Manually

Open notes.txt in a text editor (nano, vim, or VS Code). You'll see Git has marked the conflicting section like this:

```
<<<<<<< HEAD
Main branch update
=======
Feature-profile branch update
>>>>>>> feature-profile
```

Let's break down what you're looking at:

- Everything between <<<<<<< HEAD and ======= is YOUR version (what's in main right now).
- Everything between ======= and >>>>>>> feature-profile is the INCOMING version (from the branch you're merging).
- The <<<<<<, ======, and >>>>>> lines are Git's markers — they are NOT part of the actual file.

Your job is to edit the file so it looks exactly how you want it to look — keep one version, keep both, or write something entirely new. Then delete all three marker lines. For example, if you want to keep both:

```
Main branch update
Feature-profile branch update
```

Save the file once you're happy with it.

Step 4 – Commit the Merged Code

After resolving the conflict, you need to tell Git you're done and finalise the merge. Stage the resolved file and commit:

```
git add notes.txt
git commit -m "Resolve merge conflict between main and feature-profile"
```

That's it — the conflict is resolved! Git creates a merge commit that ties both histories together. Write a meaningful commit message so anyone reading the log later knows what happened.

> Always run git status before committing after a conflict. If any files still show 'both modified', you haven't resolved them yet.

Step 5 – Verify the History

Now check the full commit graph to see how everything came together:

```
git log --graph --all --oneline
```

You'll see a branching diagram in your terminal — lines forking out where you created branches and coming back together where you merged. It's a satisfying visual that shows the entire story of your project in one view.

Want even more detail? Drop --oneline to see full commit messages and authors:

```
git log --graph --all
```

What These Commands Actually Mean

- **git merge <branch>** — Brings the commits from that branch into the one you're currently on. Always be on the destination branch first.
- **Fast-forward merge** — The simplest merge — Git just moves the pointer forward because there's no diverging history. No merge commit is created.
- **Merge commit** — Created when both branches have new commits. Git combines them and records a new commit with two parents.
- **Merge conflict** — Happens when the same part of a file was changed differently on both branches. Git pauses and asks you to decide what the file should look like.
- **<<<<<<< HEAD** — Marks the start of YOUR current branch's version of the conflicting section.
- **=======** — The divider between your version and the incoming version.
- **>>>>>>> branch-name** — Marks the end of the incoming branch's version.
- **git log --graph --all --oneline** — Draws a visual ASCII graph of all commits across all branches — great for seeing the full merge history.